

React Native: Çalışma Prensipleri, Güvenlik ve Performans Rehberi

Ömer Faruk Yavuz

December 2025

Giriş

Bu doküman, React Native ile mobil uygulama geliştirirken sık sık karşımıza çıkan ancak çoğu zaman dağınık kaynaklar hâlinde bulunan temel kavramları tek bir yerde toparlamayı amaçlamaktadır. Metro paketleyicisinin rolünden dokunma ve jest sistemine, mobil uygulama güvenliğine dair pratik ilkelerden performans ve build sürelerini iyileştirme tekniklerine kadar uzanan bir yelpazede, hem teorik arka plan hem de günlük hayatta işe yarayan küçük “cheat sheet” notları sunulmaktadır.

Odak noktası, React Native geliştirme deneyimini soyut kavramlardan somut kararlara indirgemektir: Hangi durumda hangi storage çözümünü kullanmalı, hangi liste ayarları performansı etkiler, ağır işleri JS thread'ine yüklemekten animasyonları nasıl akıcı tutabiliriz, native build süresini nasıl kısaltabiliriz gibi sorulara kısa ve hatırlaması kolay cevaplar vermeye çalışır. Bu yönüyle metin, hem yeni başlayanların kavram haritası oluşturmaya hem de deneyimli geliştiricilerin günlük işlerinde hızlıca başvurabilecekleri bir referans olarak tasarlanmıştır.

.native.js ve Çoklu Platform Monorepo Mantığı

Problem

Ele alınan tipik senaryo şu biçimdedir:

- Elde **tek bir modül** bulunmaktadır: `Container`.
- Bu modülün:
 - React Native (Android ve iOS),
 - Web (React DOM),
 - ve gerekirse Node ortamında

yeniden kullanılabilmesi hedeflenmektedir.

Bununla birlikte, pratikte şu durumla karşılaşılır:

- Android ve iOS, *ortak bir React Native arayüzü* mantığını paylaşırlar,
- Web tarafında, çoğu zaman tamamen farklı bir implementasyona ihtiyaç duyulur (örneğin `<View>` yerine `<div>` kullanımı, farklı CSS yapısı vb.).

Tam bu noktada `.native.js` uzantısı devreye girer:

- Native (iOS ve Android) platformlar için **ortak bir dosya**: `Container.native.js`,
- Web platformu için ise **ayrı bir dosya**: `Container.js` tanımlanır.

Böylece, çağırılan kod tarafında import ifadesi her ortamda aynı kalır:

```
import Container from './Container';
```

Ancak çalışılan platforma bağlı olarak, ilgili bundler arka planda *farklı fiziksel dosyayı* çözümleyip projeye dahil eder.

Metro (React Native Bundler'ı) Dosyaları Nasıl Seçer?

React Native tarafında (Metro) dosya çözüm sırası kabaca şu şekildedir:

Android:

1. `Container.android.js` (varsa),
2. `Container.native.js` (varsa),
3. `Container.js`.

iOS:

1. `Container.ios.js`,
2. `Container.native.js`,
3. `Container.js`.

Dolayısıyla:

- `*.ios.js` / `*.android.js` → platforma özgü dosyalar,
- `*.native.js` → her iki native platform için ortak *fallback*,
- `*.js` → en genel fallback.

Bundan şu sonuç çıkar:

- Yalnızca native için ortak bir implementasyon istiyorsanız, `Container.native.js` yazmanız yeterlidir.
- Daha sonra Android'e özel bir davranış eklemek isterseniz, `Container.android.js` ekleyebilirsiniz; Android bu dosyayı kullanırken, iOS tarafı hâlâ `Container.native.js` üzerinden devam eder.

Web Bundler (webpack, Vite vb.) Bu Dosyalara Nasıl Bakıyor?

Web tarafında tipik bir `resolve.extensions` ayarı şu şekilde olabilir:

```
resolve: {
  extensions: ['.web.js', '.js', '.ts', '.tsx'],
}
```

Bu durumda web bundler:

1. `Container.web.js` varsa onu seçer,
2. Yoksa `Container.js` dosyasını seçer,
3. `Container.native.js` dosyasını **hiç görmez**; çünkü uzantı listesinde `.native.js` yoktur.

Bu nedenle dokümantasyonda sıkça kullanılan “bundler’ımı `.native.js` dosyalarını ignore edecek şekilde ayarla” ifadesinin pratiği aslında şudur:

- `resolve.extensions` listesine `.native.js` *eklemezsiniz*.
- Böylece web build’i `.native.js` dosyalarından haberdar bile olmaz; prod bundle gereksiz React Native kodlarıyla şişmez.

Özetle:

- **Native dünyası:** `.native.js` ile ilgilenir.
- **Web dünyası:** `.js` (veya `.web.js`) ile ilgilenir, `.native.js` yokmuş gibi davranır.

Örnek Klasör Yapısı

Aşağıda, ortak bir kütüphane için örnek klasör yapısı verilmiştir:

```
shared/
  ui/
    Container.js          # Web implementasyonu
    Container.native.js   # RN (Android + iOS) implementasyonu
  utils/
    formatDate.ts        # Her yerde aynı kullanılan pure logic
```

Container.native.js (React Native):

```
// shared/ui/Container.native.js
import React from 'react';
import { View, StyleSheet } from 'react-native';

export default function Container({ children }) {
  return <View style={styles.container}>{children}</View>;
}

const styles = StyleSheet.create({
  container: {
    padding: 16,
    backgroundColor: '#111',
  },
});
```

Container.js (Web):

```
// shared/ui/Container.js
import React from 'react';

export default function Container({ children }) {
  return (
    <div
      style={{
        padding: 16,
        backgroundColor: '#111',
        maxWidth: 600,
        margin: '0 auto',
      }}
    >
      {children}
    </div>
  );
}
```

Her iki tarafta da import ifadesi aynıdır:

```
import Container from 'shared/ui/Container';
```

- React Native uygulaması → Container.native.js,
- Web uygulaması → Container.js.

Böylece çağırın taraf platformdan tamamen habersiz kalır: “Ben sadece Container istiyorum; altında hangi platformda ne olduğunu bilmek zorunda değilim.” Bu yaklaşım, özellikle tasarım sistemi ve component library tipi yapılarda oldukça temiz bir desendir.

Node Tarafı Nerede Devreye Girer?

Node ortamı, varsayılan olarak `.native.js` uzantısını tanımaz. Zaten Node tarafında çoğunlukla UI bileşenlerini değil, *iş mantığı* (pure logic) modüllerini kullanırsınız.

Pratik bir pattern şöyle kurulabilir:

```
core/  
  domain/  
    price.ts          # pure logic, Node + Web + RN ortak  
  ui/  
    Container.js      # Web UI  
    Container.native.js# RN UI
```

- Node tarafı, örneğin

```
import { formatPrice } from 'core/domain/price';
```

şeklinde yalnızca domain/logic modüllerini kullanır; UI veya `.native` dosyalarıyla ilgilenmez.

- React Native ve Web tarafı hem domain hem ui katmanlarını kullanabilir.

Dolayısıyla `.native.js`, esas olarak **UI katmanı** için anlamlıdır; Node açısından önemli olan, iş mantığı dosyalarının platformdan bağımsız ve “pure” tutulmasıdır.

Neden Platform.select Yerine .native.js?

Her iki yaklaşım da platform farklarını yönetebilir; ancak kullanım amaçları farklıdır.

Platform.select / Platform.OS ile: Tüm mantığı tek dosyada tutarsınız:

```
const styles = {
  container: {
    backgroundColor: Platform.OS === 'ios' ? 'red' : 'green',
  },
};
```

Bu, küçük farklar için uygundur; ancak:

- Dosya büyüdükçe `if (Platform.OS === ...)` blokları çoğalarak kodu karmaşıktırabilir.
- Web ile tamamen farklı bir UI gerektiğinde, tek dosya içinde her şeyi yönetmek zor ve “kirli” bir hale gelir.
- Web bundler, React Native import’larını da görüp tree-shaking ve bundle boyutu açısından sorunlara yol açabilir.

.native.js ile:

- Web kodu ile Native kodu **dosya bazında tamamen ayrıştırılır**.
- Native uygulama:
 - Metro, yalnızca React Native import’larını ve `.native.js` dosyalarını görür.
- Web uygulama:
 - Web bundler, yalnızca DOM tabanlı kodu (`.js` veya `.web.js`) görür; RN kodlarından haberdar olmaz.

Özetle:

- **Küçük ve yerel farklar için** → `Platform.select`,
- **Tamamen farklı implementasyon / farklı UI durumlarında** → `.ios/.android/.native` dosyaları tercih edilmelidir.

“Web Bundler’ını `.native.js`’i Ignore Edecek Şekilde Ayarla” Ne Demek?

Pratik bir webpack örneği:

```
// webpack.config.js
module.exports = {
  // ...
  resolve: {
    extensions: ['.web.js', '.js', '.jsx', '.ts', '.tsx'],
  },
};
```

Burada dikkat edilmesi gereken nokta:

- Uzantılar arasında `.native.js` *yoktur*.

Dolayısıyla webpack, `import Container from './Container'` ifadesini gördüğünde:

1. Önce `Container.web.js` arar,
2. Yoksa `Container.js` arar,
3. `Container.native.js` dosyasını tamamen yok sayar.

Vite veya Rollup gibi diğer bundler’larda da benzer şekilde `resolve.extensions` veya `resolve` ayarları üzerinden aynı mantık uygulanır.

Bunun faydaları:

- Web bundle’a, React Native’e özgü component ve kodlar dahil edilmez.
- Bundle boyutu küçülür; kullanılmayan RN kodları web tarafına taşınmamış olur.

Metro ve metro.config.js

Metro, React Native projelerinde kullanılan JavaScript paketleyicisidir (bundler). Uygulamanın JavaScript kodlarını ve varlıklarını (örneğin resimler, fontlar vb.) bir araya getirerek cihaza iletir. Web dünyasındaki Webpack veya Vite ne ise, React Native tarafında da aynı görevi Metro üstlenir. Proje kök dizininde yer alan `metro.config.js` dosyası, Metro'nun davranışını özelleştirmek için kullanılır.

Gesture Responder System

Gesture Responder System, React Native uygulamalarında dokunma ve jest (*gesture*) yaşam döngüsünü yöneten düşük seviye mekanizmadır. Tek veya çoklu dokunuşlar sırasında:

- Kaydırma (scroll),
- Sürükleme (drag / pan),
- Basma (*tap*)

gibi etkileşim türlerini ayırt ederek, hangi bileşenin ilgili dokunuşa *responder* (dokunuşun sahibi) olacağına karar verilmesini sağlar. Bu karar, bileşen hiyerarşisini doğrudan bilmeden, bileşenlerin kendi aralarında protokol üzerinden “anlaşması” ile verilir.

Yüksek Seviye Kullanım: Touchable* ve Pressable

Düşük seviye responder API’leri karmaşık olduğundan, React Native günlük kullanım için aşağıdaki yüksek seviye soyutlamaları sunar:

- `TouchableOpacity`, `TouchableHighlight`,
- `Pressable` vb.

Bu bileşenler, Gesture Responder System üzerine inşa edilmiş “dokunulabilir” (*tappable*) arayüz elemanlarıdır. Web tarafında buton/link kullanılan yerlerde, React Native projelerinde genellikle bu bileşenler tercih edilir. Böylece:

- Görsel geri bildirim (highlight, opacity değişimi),
- İşlemi iptal edebilme (parmağı kaydırarak vazgeçme)

gibi davranışlar tutarlı ve otomatik şekilde sağlanır.

İyi Kullanıcı Deneyimi İçin Temel İlkeler

Gesture Responder System üzerine kurulu bileşenler tasarlanırken özellikle iki ilkeye dikkat edilmelidir:

1. **Geri bildirim (feedback):** Kullanıcı dokunduğunda hangi elemanın tepki verdiği ve parmağını çektiğinde ne olacağı, renk değişimi, ölçek (*scale*) veya benzeri efektlerle açıkça gösterilmelidir.
2. **İptal edilebilirlik:** Kullanıcı parmağını etkileşim bölgesinin dışına kaydırarak işlemi iptal edebilmelidir. Bu, yanlışlıkla dokunmalara karşı güven hissi sağlar.

Düşük Seviye Yaşam Döngüsü ve PanResponder

Daha özel drag/sürükleme senaryolarında, düşük seviye yaşam döngüsü call-back'leri kullanılarak bir bileşenin responder olup olmayacağı belirlenebilir:

- `onStartShouldSetResponder`, `onMoveShouldSetResponder`: Dokunuşu sahiplenme isteğinin belirtildiği aşamalar.
- `onResponderGrant`, `onResponderMove`, `onResponderRelease`: Dokunuş verisini aktif olarak işleyen aşamalar.
- `onResponderTerminationRequest`, `onResponderTerminate`: Dokunuşun başka bir bileşene devredilmesi veya iptal edilmesi ile ilgili aşamalar.

Bu düşük seviye akışla doğrudan çalışmak yerine, React Native çekirdeğinde yer alan **PanResponder** aracı sıklıkla tercih edilir. **PanResponder**, başlangıç/hareket / bırakma mantığını paketleyerek özellikle:

- Kart sürükleme,
- Basit swipe veya pan jestleri,
- Özel slider bileşenleri

gibi etkileşimlerde daha ergonomik bir API sunar.

Modern Yaklaşım: react-native-gesture-handler

Daha karmaşık, performans kritik veya çok sayıda jestin kullanıldığı ürün seviyesindeki uygulamalarda, topluluk tarafından yaygın olarak:

- **react-native-gesture-handler** (jest algılama),
- Reanimated (animasyonların UI thread üzerinde çalıştırılması)

yığını tercih edilmektedir. Bu yığın, Gesture Responder System'in üzerine kurulu olmakla birlikte, platformun yerel (*native*) jest sistemleriyle daha sıkı entegre çalışır ve modern “drag / swipe / bottom sheet” benzeri arayüzlerin geliştirilmesinde fiili standart haline gelmiştir.

Mobil Uygulama Güvenliđi için Kısa Rehber

Mobil tarafta hiçbir uygulama mutlak anlamda “tam güvenli” değildir. Güvenlik, her zaman bir risk azaltma ve maliyet dengesi meselesidir: Verinin ne kadar hassas olduđu, kaç kullanıcının etkileneceđi ve bir ihlal durumunda oluşacak zararın boyutu, hangi önlemlerin alınması gerektiđini belirler. Harcanan emek ve disiplin arttıkça, saldırganın işini zorlaştıran katman sayısı da artar.

Hassas bilgiler söz konusu olduğunda, API key ve secret gibi değerlerin asla doğrudan uygulama koduna gömülmemesi gerekir. **react-native-config** veya **react-native-dotenv** benzeri kütüphaneler, derlenmiş bundle içinden hala okunabildiđi için bu değerleri gerçekten “gizlemez”; yalnızca yönetimini kolaylaştırır. Doğru model, her zaman bir *orchestration* katmanını kullanmaktır: istemci, önce sizin kontrolünüzdeki backend’e veya serverless fonksiyona istek gönderir; asıl üçüncü parti API’lerle konuşan ve secret’ları tutan taraf da yalnızca bu sunucu olur. Böylece client, kritik bilgilere hiçbir zaman doğrudan erişemez.

Verinin saklanma şekli de en az ne saklandıđı kadar önemlidir. Diske yazılan ve uygulama kapandıktan sonra da kalan *persisted* veriler, kullanım açısından konforlu olsa da saldırı yüzeyini büyütür. Sadece RAM’de tutulan *unpersisted* veriler ise daha güvenli olmakla birlikte, her uygulama açılışında yeniden elde edilmeyi gerektirir. React Native ekosisteminde yaygın olarak kullanılan AsyncStorage, web’deki **localStorage** hissine yakın, pratik fakat güvenlik açısından sınırlı bir çözümdür. Kullanıcı tercihleri, tema ayarları veya önemsiz bayraklar için uygundur; ancak erişim token’ları, şifreler, secret’lar ve benzeri hassas veriler için tercih edilmemelidir.

Gerçekten hassas bilgiler için, platformların sunduđu güvenli depolama altyapıları kullanılmalıdır. iOS tarafında Keychain Services, Android tarafında ise Encrypted Shared Preferences ve Keystore mekanizmaları bu amaçla tasarlanmıştır. JavaScript katmanından erişim için **expo-secure-store** veya **react-native-keychain** gibi kütüphaneler kullanılabilir. Bunun yanında, Redux state’ini veya benzer global durumları persist ederken yanlışlıkla token, kredi kartı bilgisi veya kişisel veri gibi alanları AsyncStorage’a yazmamak; ayrıca loglama, crash raporlama (Sentry, Crashlytics vb.) gibi kanallara da bu tip verileri sızdırmamak kritik önemdedir.

Kimlik doğrulama ve deep linking tarafında da benzer hassasiyet gereklidir. **myapp://products/1** gibi scheme tabanlı deep link’ler, mobil işletim sistemlerinde merkezi bir kayıt otoritesine bađlı olmadıkları için başka bir uygulama tarafından da aynı scheme ile “sahiplenilebilir”. Bu nedenle, deep link üzerinden erişim token’ı, şifre veya benzeri gizli bilgiler taşımak güvenli kabul edilmez; yalnızca içerik kimliđi, slug veya benzeri non-sensitive veriler bu yolla iletilmelidir. iOS’ta universal links kullanmak, içerik odaklı yönlendirmelerde ek bir güven katmanını sağlar.

OAuth2 akışları söz konusu olduğunda, özellikle mobil ortamda URL scheme çatışmaları ve yönlendirme zafiyetleri nedeniyle PKCE (Proof of Key Code Exchange) mekanizması neredeyse zorunlu bir ek güvenlik katmanını haline gelmiştir. PKCE, yetkilendirme kodunu alan istemci ile token talep eden istemcinin gerçekten

aynı olduğunun kanıtlanmasını hedefler.

İstemci tarafında önce rastgele bir `code_verifier` üretilir ve bunun SHA-256 ile özetlenmiş hâli `code_challenge` olarak yetkilendirme (`/authorize`) isteğine eklenir. Kimlik sağlayıcı, daha sonra `/token` isteğiyle gelen `code_verifier` değerinden yeniden bir `code_challenge` hesaplar ve bunu, ilk aşamada sakladığı `code_challenge` ile karşılaştırır; eşleşme yoksa access token üretmez. Bu sayede, yetkilendirme kodu saldırganın eline geçmiş olsa bile, ilgili `code_verifier` olmadan tek başına kullanılamaz. React Native ekosisteminde `react-native-app-auth` gibi kütüphaneler, bu PKCE tabanlı OAuth2 akışını güvenli ve pratik şekilde uygulamaya almayı kolaylaştırır.

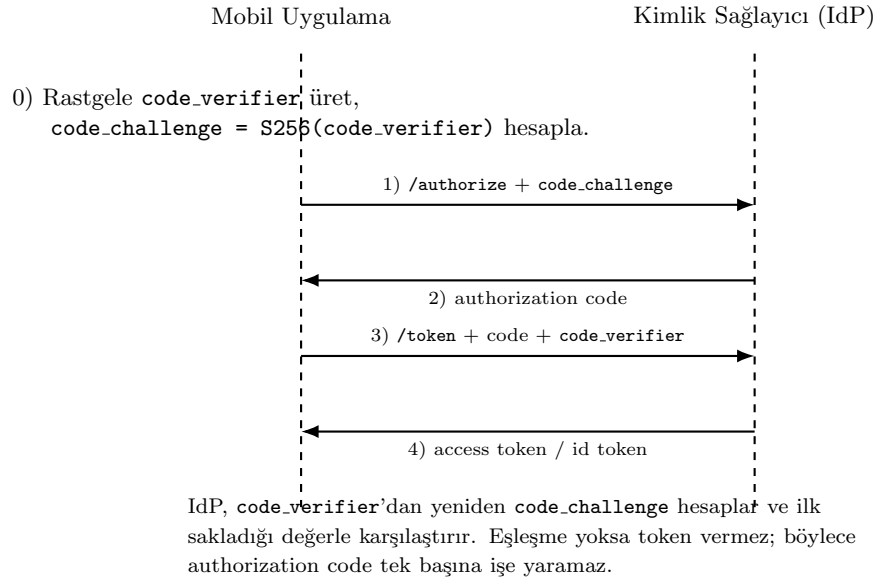


Figure 1: OAuth2 PKCE akışının şematik gösterimi.

Ağ katmanında temel ilke, tüm isteklerin her zaman `https://` üzerinden yapılmasıdır; aksi halde veriler iletim sırasında düz metin olarak taşınır. Buna ek olarak, daha ileri seviye bir önlem olarak SSL pinning kullanılabilir. Bu yöntem, istemci uygulamaya belirli sertifikaları veya public key'leri özellikle "güvenilir" olarak gömecek şekilde yapılandırma yapar; sunucunun sertifikası bu listeye uyuşmadığında bağlantı reddedilir ve klasik MITM senaryoları zorlaşır. Ancak sertifikaların belirli aralıklarla yenilendiği unutulmamalıdır; pin'lenen sertifika değiştiğinde, uygulamanın da güncellenmesi gerekir ve eski sürümler güncelleme yapılana kadar bağlantı kuramaz hale gelebilir.

Sonuç olarak, iyi bir mobil güvenlik yaklaşımı şu ilkelere dayanır: Secret'ları uygulama koduna gömmemek, hassas veriyi yalnızca güvenli storage mekanizmalarında tutmak, deep link'ler üzerinden asla token veya kişisel veri taşımamak, OAuth2 kullanılıyorsa mümkünse PKCE ile desteklemek, tüm trafiği HTTPS

üzerinden geçirmek ve gerek duyulduğunda SSL pinning gibi ek katmanlarla güçlendirmek. Ve en temel prensip: Toplamadığınız ve saklamadığınız veri, sızması mümkün olmayan tek veridir.

React Native’de Performans

React Native’de performansın temel hedefi, ekranda saniyede en az 60 kare üreterek arayüzün “native” hissini korumaktır. Bu, her frame için yaklaşık 16,67 ms süreniz olduğu anlamına gelir. Bu kısa süre içinde hem JavaScript tarafındaki hesaplamalar hem de native view güncellemeleri tamamlanamazsa kareler düşer, kullanıcı da takılma ve gecikme hisseder.

JS Thread ve UI Thread Farkı

JavaScript thread’i; React bileşenlerinin render edilmesi, state güncellemeleri, iş mantığı ve API çağrılarının yürütüldüğü yerdir. Bu thread uzun süre meşgul olduğunda animasyonlar donar, dokunma olayları geç yanıt verir ve uygulama ağır hissedilir. Büyük bir component ağacında pahalı bir `setState` dalgası 200 ms sürdüğünde, arka arkaya birçok frame kaybedebilirsiniz.

UI (main) thread ise native tarafın sorumluluğundadır. Özellikle `native-stack` ile yapılan navigation animasyonları ve `ScrollView` gibi native scroll bileşenleri burada çalışır. Bu nedenle, JS tarafı kısa süreli kilitlense bile sayfa geçişleri veya kaydırma hareketi görece akıcı kalabilir; yalnızca scroll event’lerinin JS’ye iletilmesi gecikir.

Yaygın Performans Sorunları ve Kısa Çözümler

Geliştirme modunda (`dev = true`) ek kontrol ve uyarılar nedeniyle JS thread’i belirgin biçimde yavaşlar; bu yüzden gerçek performans her zaman release build üzerinde ölçülmelidir.

Uygulama çapında yoğun `console.log` kullanımı da, production bundle’da bile gereksiz iş yükü oluşturabilir; bunu engellemek için `babel-plugin-transform-remove-console` gibi araçlar kullanmak faydalıdır.

Büyük listelerde performans problemlerinin sık görülen nedeni, her öğenin ölçülmesinin pahalı olmasıdır. `FlatList` kullanırken `getItemLayout` tanımlamak bu maliyeti ciddi biçimde azaltır; daha uç senaryolarda `FlashList` veya benzeri performans odaklı liste kütüphaneleri tercih edilebilir.

Aynı şekilde, JS thread’e aynı anda çok iş bindirmek (navigator geçişi sırasında ağır hesaplar, geniş re-render dalgaları) hem animasyonları hem dokunma tepkilerini bozar; bu tip işleri `InteractionManager` ile animasyon sonrasına ertelemek, gerektiğinde `LayoutAnimation` ve `Animated+useNativeDriver` gibi native tabanlı animasyon mekanizmalarından yararlanmak daha sağlıklı bir yaklaşımdır.

Örnek: Navigasyon ve Ağır İş Aynı Anda Yapılır

Aşağıda, bir kart bileşenine tıklandığında hem başka bir ekrana navigasyon yapılması hem de arkada ağır bir hesaplama çalıştırılması durumu gösterilmektedir. Aynı frame içinde hem navigasyon hem de ağır işi tetiklemek, JS thread'i kilitleyerek animasyon ve dokunma tepki süresini bozar. Bunu daha sağlıklı hale getirmek için ağır işi `InteractionManager` ile animasyonlar sonrasına ertelemek ve animasyonları `useNativeDriver` ile native tarafa taşımak daha doğru bir yaklaşımdır.

Kötü Yaklaşım: Her Şeyi Aynı Anda Çalıştırmak

```
function HomeScreen({ navigation }) {
  const handleCardPress = () => {
    // 1) Hemen ağır iş
    doExpensiveSyncWork(); // 150{200 ms sürsün

    // 2) Sonra navigasyon
    navigation.navigate('Details');
  };

  return (
    <TouchableOpacity onPress={handleCardPress}>
      <Text>Detaylara git</Text>
    </TouchableOpacity>
  );
}

function doExpensiveSyncWork() {
  // Simülasyon: büyük bir hesap, JSON parse, vs.
  const start = Date.now();
  while (Date.now() - start < 200) {
    // busy loop
  }
}
```

Bu senaryoda `onPress` tetiklenir tetiklenmez JS thread yaklaşık 200 ms boyunca meşgul olur. Bu süre içinde hem `Touchable` bileşeninin feedback animasyonu gecikir hem de `navigation.navigate` çağrısı gecikmeli olarak işlenir; kullanıcı “bastım ama geç tepki veriyor” hissine kapılır.

Daha Sağlıklı Yaklaşım: Önce Feedback ve Navigasyon, Sonra Ağır İş

```
import {
  InteractionManager,
  LayoutAnimation,
  Platform,
  UIManager,
} from 'react-native';
import Animated, { Easing } from 'react-native-reanimated'; // veya klasik Animated

// Android'de LayoutAnimation için:
if (Platform.OS === 'android' && UIManager.setLayoutAnimationEnabledExperimental) {
  UIManager.setLayoutAnimationEnabledExperimental(true);
}

function HomeScreen({ navigation }) {
  const scale = useRef(new Animated.Value(1)).current;

  const handleCardPress = () => {
    // 1) Önce küçük, hızlı bir native animasyon (useNativeDriver)
    Animated.timing(scale, {
      toValue: 0.97,
      duration: 120,
      easing: Easing.out(Easing.quad),
      useNativeDriver: true, // animasyon UI thread'e kayıyor
    }).start(() => {
      Animated.timing(scale, {
        toValue: 1,
        duration: 120,
        easing: Easing.out(Easing.quad),
        useNativeDriver: true,
      }).start();
    });

    // 2) Navigasyonu hemen tetikle (native-stack ise animasyon UI thread'de)
    navigation.navigate('Details');

    // 3) Ağır işi, geçiş ve animasyonlar bittikten SONRA yap
    InteractionManager.runAfterInteractions(() => {
      doExpensiveSyncWork(); // Artık frame öldürme riski çok daha az
    });
  };

  return (
    <Animated.View style={{ transform: [{ scale }] }}>
```

```

        <TouchableOpacity onPress={handleCardPress}>
            <Text>Detaylara git</Text>
        </TouchableOpacity>
    </Animated.View>
);
}

function doExpensiveSyncWork() {
    // Simülasyon: ağır iş
    const start = Date.now();
    while (Date.now() - start < 200) {
        // heavy work
    }

    // Örn: layout değişimi varsa, LayoutAnimation ile yumuşat:
    LayoutAnimation.configureNext(LayoutAnimation.Presets.easeInEaseOut);
    // setState(...) vs.
}

```

Bu yaklaşımda:

- Küçük ölçek (*scale*) animasyonu **useNativeDriver: true** ile UI thread'e taşındığı için, JS tarafındaki kısa gecikmelerden daha az etkilenir.
- Navigasyon hemen tetiklenir; özellikle **native-stack** kullanıldığında sayfa geçiş animasyonu native tarafta akıcı şekilde çalışır.
- **InteractionManager.runAfterInteractions** sayesinde ağır iş, kullanıcı animasyonu ve geçişi gördükten sonra, etkileşim tamamlandığında başlatılır.
- **LayoutAnimation** kullanımı, state değişimi sonrası layout atlamalarını yumuşatarak, “fire-and-forget” türü basit animasyonların native tarafta yönetilmesini sağlar.

Bu örnek, JS thread'e yük bindiren işleri doğrudan **onPress** içinde çalıştırmak yerine, önceliği her zaman kullanıcıya verilen görsel feedback ve akıcı navigasyona bırakmanın ve ağır işleri etkileşim sonrası zamana yaymanın performans açısından ne kadar kritik olduğunu göstermektedir.

UI FPS düşüşleri çoğu zaman görsel bileşenlerin çizim maliyetinden kaynaklanır. Özellikle Android’de transparan metin ve görsellerin üst üste bindirildiği, alpha compositing gerektiren durumlar pahalıdır. Bazı bileşenler için `renderToHardwareTextureAndroid` kullanmak bu maliyeti azaltabilir, ancak aşırı kullanım bellek tüketimini yükseltir; profil ederek ve iş bittikten sonra kapatmak gerekir. Benzer şekilde, bir görselin `width/height` özelliğini her frame’de değiştirmek, özellikle iOS’ta her seferinde yeniden scale/crop işlemi tetiklediği için pahalıdır; bunun yerine `transform: { scale }` ile yapılan zoom/resize animasyonları çok daha hafiftir.

Örnek: Çizim Maliyeti, `renderToHardwareTextureAndroid` ve `scale` Animasyonu

UI tarafındaki FPS düşüşleri çoğu zaman, ekranda görülen bileşenlerin nasıl çizildiği (render edildiği) ile ilgilidir. Özellikle Android’de transparan metin ve görsellerin üst üste geldiği, alpha compositing gerektiren sahneler pahalıdır. Aşağıdaki örnekler, bu maliyeti nasıl yönetebileceğimize dair somut bir fikir verir.

1) `renderToHardwareTextureAndroid` ile Pahalı Bileşeni “Önceden” Çizmek

Örneğin, arka planda bulanık bir görsel, üstünde yarı saydam bir overlay ve onun da üzerinde text bulunan bir kart düşünelim. Android bu tür bir yapıyı her frame yeniden compositing yaparak çizer; animasyon sırasında maliyet artar.

```
function FancyCard() {
  return (
    <View style={styles.cardContainer}>
      <Image
        source={require('./bg.jpg')}
        style={styles.bgImage}
        blurRadius={8}
      />
      <View style={styles.overlay} />
      <Text style={styles.title}>Merhaba dünya</Text>
    </View>
  );
}

const styles = StyleSheet.create({
  cardContainer: {
    width: 320,
    height: 180,
    borderRadius: 16,
    overflow: 'hidden',
  },
  bgImage: {
    ...StyleSheet.absoluteFillObject,
```

```

    },
    overlay: {
        ...StyleSheet.absoluteFillObject,
        backgroundColor: 'rgba(0,0,0,0.3)',
    },
    title: {
        position: 'absolute',
        bottom: 16,
        left: 16,
        color: 'white',
        fontSize: 18,
        fontWeight: '600',
    },
});

```

Bu kartı listede yatay olarak kaydırırken veya `scale` animasyonu uygularken FPS düşüşleri görebilirsiniz. Çok pahalı bir kartınız varsa, Android tarafında bu kartı bir defa GPU üzerinde “texture” olarak hazırlayıp tekrar kullanmak için `renderToHardwareTextureAndroid` özelliğini seçici olarak açabilirsiniz:

```

function FancyCard() {
    return (
        <View
            style={styles.cardContainer}
            renderToHardwareTextureAndroid={true} // sadece gerçekten pahalıysa
        >
            <Image
                source={require('./bg.jpg')}
                style={styles.bgImage}
                blurRadius={8}
            />
            <View style={styles.overlay} />
            <Text style={styles.title}>Merhaba dünya</Text>
        </View>
    );
}

```

Bu sayede Android, kartı her frame yeniden compositing yapmak yerine, hazırlanmış bir texture üzerinden daha ucuz şekilde ekrana çizebilir. Ancak:

- Her kart için bu özelliği açmak, özellikle büyük listelerde ciddi bellek tüketimine yol açabilir.
- Bu nedenle, yalnızca gerçekten pahalı birkaç bileşende, profil sonuçlarına bakarak ve işiniz bittiğinde bu bayrağı kapatarak kullanmak iyi bir pratiktir.

2) **width/height ile Animasyon vs. transform: { scale }** Bir görseli büyütüp küçültmek istediğinizde, iki temel yaklaşımınız vardır:

1. Her frame'de **width / height** değerlerini değiştirmek.
2. Görselin boyutunu sabit tutup, **transform: [scale]** ile ölçeklemek.

Kötü yaklaşım (pahalı: her frame yeniden layout ve scale/crop):

```
function ScalingImageBad() {
  const [size, setSize] = useState(100);

  useEffect(() => {
    let current = 100;
    const id = setInterval(() => {
      current = current === 100 ? 150 : 100;
      setSize(current);
    }, 300);
    return () => clearInterval(id);
  }, []);

  return (
    <Image
      source={require('./photo.jpg')}
      style={{ width: size, height: size, borderRadius: 8 }}
      resizeMode="cover"
    />
  );
}
```

Bu tarz bir kullanımda, özellikle iOS'ta her boyut değişiminde görselin yeniden scale/crop edilmesi gerekebilir. Bu, animasyon süresince ek maliyet anlamına gelir.

Daha sağlıklı yaklaşım (hafif: transform ile ölçekleme):

```
import { Animated, Easing } from 'react-native';

function ScalingImageGood() {
  const scale = useRef(new Animated.Value(1)).current;

  useEffect(() => {
    const anim = Animated.loop(
      Animated.sequence([
        Animated.timing(scale, {
          toValue: 1.5,
          duration: 300,
          easing: Easing.out(Easing.quad),
          useNativeDriver: true,    // animasyon UI thread'de
        }),
        Animated.timing(scale, {
          toValue: 1,
          duration: 300,
          easing: Easing.out(Easing.quad),
          useNativeDriver: true,
        }),
      ])
    );
    anim.start();
    return () => anim.stop();
  }, [scale]);

  return (
    <Animated.Image
      source={require('./photo.jpg')}
      style={{
        width: 120,
        height: 120,
        borderRadius: 8,
        transform: [{ scale }],
      }}
      resizeMode="cover"
    />
  );
}
```

Burada:

- Görselin temel boyutu (**width**, **height**) sabit, layout hesapları değişmiyor.
- Görünen büyüme/küçülme, yalnızca **transform: [scale]** üzerinden yapılıyor; **useNativeDriver: true** ile animasyon olabildiğince UI thread'e kaydırılıyor.
- Özellikle iOS'ta her frame yeniden crop/scale işi yapılmadığı için, FPS ve enerji tüketimi açısından daha verimli bir çözüm elde ediliyor.

Bu örnekler, UI FPS düşüşlerinin sadece “JS yavaş” olduğu için değil, ekranın nasıl çizildiğiyle de yakından ilgili olduğunu; doğru yerde **renderToHardwareTextureAndroid** kullanmanın ve boyut animasyonlarını **transform** üzerinden çözmenin performans açısından ne kadar kritik olduğunu göstermektedir.

React Native Build Hızlandırma Rehberi

Büyük React Native projelerinde özellikle Android tarafında native build süreleri dakikalara çıkabilir. Aşağıdaki teknikler, geliştirme ortamında build sürelerini kısaltmak için kullanılabilir, nispeten ileri seviye (yarı-native) optimizasyonlardır.

1. Sadece Tek ABI Derlemek (Android)

Android, varsayılan olarak dört mimari için derleme yapar: `armeabi-v7a`, `arm64-v8a`, `x86`, `x86_64`. Geliştirme sırasında genellikle tek bir emulator veya tek bir fiziksel cihaz kullandığınız için, yalnızca aktif mimariyi derlemek ciddi hız kazandırır.

React Native CLI ile:

```
yarn react-native run-android --active-arch-only
```

Doğrudan Gradle ile:

```
./gradlew :app:assembleDebug -PreactNativeArchitectures=x86,x86_64
```

gradle.properties üzerinden varsayılan ayar:

```
# Hepsini build et (default)
reactNativeArchitectures=armeabi-v7a,arm64-v8a,x86,x86_64
```

Release APK/AAB üretirken tüm ABI'lar için derleme yapmayı unutmamak gerekir; aksi halde bazı cihazlarda uygulama çalışmayabilir.

2. Gradle Configuration Cache (Android)

Gradle build süreci, *configuration* (tüm `build.gradle` dosyalarının okunması ve yapının kurulması) ve *execution* (derleme ve task'lerin çalıştırılması) fazlarından oluşur. Configuration cache etkinleştirildiğinde, ilk build'te yapılan konfigürasyon hesaplaması cache'e yazılır ve sonraki build'lerde bu faz büyük ölçüde atlanır.

android/gradle.properties içine:

```
org.gradle.configuration-cache=true
```

Özellikle Android/native tarafa sık sık dokunulan projelerde önemli zaman kazancı sağlayabilir.

3. Maven Mirror Kullanmak (Android)

Gradle, bağımlılıkları default olarak internet üzerinden (Maven Central vb.) indirir. Kurumsal ortamlarda, şirket içinde bir Maven proxy/mirror (Artifactory, Nexus vb.) kullanmak hem hızlı hem de daha stabil olabilir.

android/gradle.properties örneği:

```
# Hermes örnek ayar
hermesEnabled=true

# Tüm dependency'leri sadece şu Maven repo'dan çek:
exclusiveEnterpriseRepository=https://my.internal.proxy.net/
```

Bu sayede bağımlılıklar doğrudan iç ağ üzerindeki mirror'dan alınır.

4. Derleyici Cache'i Kullanmak (Native Kod)

Native kısımda (C++, Obj-C, Swift) sık derleme yapılıyorsa, derleyici cache'leri ciddi hız kazandırır.

a) Lokal cache: ccache

- Aynı input için derleme çıktısını cache'ler; dosya değişmediyse derleme adımı atlanır.

Kurulum (macOS):

```
brew install ccache
```

İstatistik:

```
ccache -s          # istatistik
ccache --zero-stats
ccache --clear
```

iOS + Xcode için, ios/Podfile içinde:

```
post_install do |installer|
  react_native_post_install(
    installer,
    config[:reactNativePath],
    :mac_catalyst_enabled => false,
    # TODO: Uncomment the line below
    :ccache_enabled => true
  )
end
```

CI ortamında ~/Library/Caches/ccache klasörünü save/restore etmek mümkündür; bozulmuş cache riskine karşı zaman zaman tam temiz build yapmak iyi bir pratiktir.

FlatList Performans Cheat Sheet

Temel Kavramlar

FlatList, altında **VirtualizedList** motorunu kullanarak büyük listeleri verimli şekilde render eder. Performansla ilgili temel kavramlar şunlardır:

- **VirtualizedList:** FlatList'in üzerinde çalıştığı sanallaştırılmış liste altyapısıdır; yalnızca görünür pencereyi ve yakın çevresini render eder.
- **Memory consumption:** Aynı anda bellekte tutulan item sayısı ve bu item'lara ait view verisinin toplam maliyetidir; çok yükselirse crash riski artar.
- **Responsiveness:** Kullanıcının dokunuş, kaydırma gibi etkileşimlerine ne kadar hızlı yanıt verildiğidir.
- **Blank areas:** Scroll sırasında liste henüz render edemediği için ekranda boş gri/beyaz alanların görülmesidir.
- **Viewport:** Ekranda o anda fiilen görünen bölgedir.
- **Window:** FlatList'in bellekte tuttuğu görünür pencere; viewport'tan geniş bir alan kapsar (üst ve alt buffer).

Önemli FlatList / VirtualizedList Props

removeClippedSubviews Viewport dışındaki item'ları native view hiyerarşisinden *detach* eder. Ana thread üzerindeki yükü azalttığı için dropped frame sayısını düşürebilir; ancak özellikle iOS'ta **transform** ve **position: absolute** kombinasyonlarında içerik kaybolması gibi yan etkilere yol açabilir. Bellek kazancı sınırlıdır; view'lar detach edilir ama tamamen deallocate edilmez.

maxToRenderPerBatch (varsayılan: 10) Her batch'te kaç item'ın render edileceğini belirler. Değeri artırmak, scroll sırasında boş alan görme olasılığını azaltır (fill rate artar); ancak aynı anda daha fazla iş JS thread'e yükleneceği için responsiveness düşebilir.

updateCellsBatchingPeriod (varsayılan: 50 ms) Batch'ler arasındaki bekleme süresini (milisaniye) tanımlar; yani yeni batch'lerin ne kadar sık tetikleneceğini kontrol eder. **maxToRenderPerBatch** ile birlikte kullanılarak “daha sık, az item” ya da “daha seyrek, çok item” gibi stratejiler kurulabilir. Çok seyrek batch, blank area riskini artırırken; çok sık batch JS thread'i tıkararak responsiveness'i olumsuz etkileyebilir.

initialNumToRender (varsayılan: 10) İlk render'da kaç item'ın çizileceğini tanımlar. Bu değeri, ekranı baştan tam dolduracak şekilde ayarlamak ilk yükleme performansına büyük katkı sağlar. Çok düşük tutulursa daha ilk ekranda dahi boş alanlar görülebilir.

windowSize (varsayılan: 21) 1 bir viewport yüksekliğini ifade eder; 21 değeri, görünür pencerenin üstünde 10, altında 10 ve ortada 1 viewport'luk alanın bellekte tutulduğu anlamına gelir. Büyük **windowSize** scroll sırasında boş alan görme ihtimalini azaltır fakat daha fazla item'ı aynı anda mount ettiği için bellek tüketimini artırır; küçük **windowSize** tersine bellek kazandırır ancak blank area riskini yükseltir.

List Item'lar İçin İpuçları

Basit ve Hafif Component Tasarımı Liste item bileşeni, uygulamanın performans kalbidir. Çok katmanlı, yoğun iş mantığı barındıran, ağır stillerle donatılmış item'lar render süresini yükseltir. Büyük listeler için, görece sadeleştirilmiş, sadece listede gösterilecek özet bilgileri içeren özel item component'leri tasarlamak performansa önemli katkı sağlar; detaylı içerik ayrı bir "item detail" ekranına taşınabilir.

React.memo ile Gereksiz Render'ları Azaltma Item component'lerini **React.memo** ile sarmalamak, yalnızca props değiştiğinde yeniden render olmalarını sağlar:

```
const MyListItem = memo(
  ({ title }: { title: string }) => (
    <View>
      <Text>{title}</Text>
    </View>
  ),
  (prev, next) => prev.title === next.title
);
```

Karşılaştırma fonksiyonu **true** döndüğünde, **FlatList** aynı item'ı yeniden render etmez; özellikle büyük listelerde bu teknik ciddi tasarruf sağlar.

Optimize / Cached Görsel Kullanımı Her **Image** bileşeni yeni bir instance ve yük bindirir. Görseller ne kadar hızlı yüklenir ve cache'ten okunursa, JS thread o kadar çabuk boşalır. [@d11/react-native-fast-image](#) veya benzeri optimize image bileşenleri kullanarak caching ve daha hızlı görüntüleme elde etmek mümkündür; listede küçük thumbnail'ler, detay ekranda tam boy görseller göstermek iyi bir desendir.

getItemLayout Kullanımı (Sabit Boyutlu Item'lar) Tüm item'lar aynı yükseklik/genişliğe sahipse, **getItemLayout** tanımlayarak **FlatList**'in her item için **async** ölçüm yapmasını engelleyebilirsiniz. Bu, scroll ve ilk render performansını gözle görülür biçimde iyileştirir. Item boyutları dinamik ama performans kritikse, tasarım ekibiyle görüşüp sabit yükseklikli bir yapı benimsemek çoğu zaman daha sürdürülebilir bir optimizasyondur.

keyExtractor ve Stabil Key'ler Her item'a stabil ve benzersiz bir key verilmesi (`keyExtractor={item => item.id}`) React'in yeniden sıralama ve diff işlemlerinde daha az gereksiz render yapmasını sağlar. Key'lerin index'e bağlı olması, özellikle insert/delete durumlarında büyük re-render dalgalarına yol açabilir.

renderItem İçinde Anonim Fonksiyondan Kaçınma `renderItem` fonksiyonunun her render'da yeniden tanımlanması, memo'lanmış item component'lerinin bile gereksiz yere yeniden render edilmesine neden olabilir. Fonksiyonu component dışında veya `useCallback` ile tanımlamak daha doğrudur:

```
const renderItem = useCallback(
  ({ item }) => (
    <MyListItem title={item.title} />
  ),
  []
);

return <FlatList data={items} renderItem={renderItem} />;
```

Class component'lerde ise `renderItem`'i `render()` dışına, sınıf metodu olarak taşımak benzer bir etki yaratır.

JS Yükleme Performansı için Mini Not

JavaScript kodunun parse edilmesi ve çalıştırılması hem zaman hem de bellek tüketir. Uygulama büyüdükçe, gereksiz kodu başlangıçta yüklemek yerine ihtiyaç oldukça (lazy) yüklemek, özellikle açılış süresi ve ilk etkileşim performansı açısından büyük fayda sağlar. React Native, belirli optimizasyonları zaten sunar; fakat doğru ayarlarla bu sürece siz de aktif katkıda bulunabilirsiniz.

1. Hermes Kullanım (Önerilen)

Hermes, yeni React Native projelerinde varsayılan JavaScript engine olarak gelir. Özellikle release build’lerde:

- JS kodunu önceden bytecode’a çevirir (AOT).
- Bytecode’u ihtiyaç oldukça belleğe yükler.
- Her seferinde plain JS parse etmek yerine bytecode üzerinden çalıştığı için startup süresi belirgin biçimde kısalmır.

2. Büyük Component’leri Lazy-Load Edin

Ekran (screen) seviyesindeki ağır component’ler veya ilk açılışta görülme olasılığı düşük, kalın bağımlılıklara sahip modüller, **React.lazy** ve **Suspense** ile lazy-load edilmelidir. Böylece bu component’lerin kodu, gerçekten ilk kez render edilene kadar bundle’dan çekilmez; yeni ekranlar eklendiğinde dahi uygulama açılış süresi daha az etkilenir.

3. Module Side Effect’lerinden Kaçınım

Lazy-load ve inline **require**, modüllerin çalıştırılma sırasını değiştirir. Eğer bir modül:

- global seviyesinde değişiklik yapıyor,
- event’lere subscribe oluyor,
- polyfill veya patch uyguluyorsa,

bu davranışlar *side effect* olarak değerlendirilebilir. Bu tür side effect’ler, modülü lazy-load etmeye başladığınızda, beklenmedik bug’lara veya crash’lere yol açabilir. İdeal olan, modüllerin yalnızca değer export etmesi; global etkilerin ise component lifecycle içinde, kontrollü noktalarda çalıştırılmasıdır.

4. Inline require() ile Elle Geciktirme

Klasik import yerine `require()`'ı tetikleyici bir olay (örneğin `onPress`) anında çağırarak modül yüklenmesini manuel olarak geciktirebilirsiniz.

Normal kullanım:

```
import VeryExpensive from './VeryExpensive';
```

Yerine:

```
let VeryExpensive: any = null;

const didPress = () => {
  if (!VeryExpensive) {
    VeryExpensive = require('./VeryExpensive').default;
  }
  setNeedsExpensive(true);
};
```

Böylece dosya başında import etmek yerine, modül ilk kez gerçekten ihtiyaç duyulduğunda yüklenir. Bu yöntem senkron çalışır; `React.lazy / import()` gibi async değildir.

5. Otomatik Inline require (React Native CLI)

React Native CLI kullanıyorsanız, `require()` çağrıları (import'lar değil) otomatik olarak inline edilir; yani modül, ancak gerçekten `require` çağrısı yapıldığında eval edilir:

```
const VeryExpensive = require('./VeryExpensive').default;
// ilk require anına kadar modül eval edilmez
```

Expo projelerinde bu özellik varsayılan olarak kapalıdır; Metro ayarlarında `inlineRequires: true` ile açılabilir.

Dikkat: Inline Require Tuzakları

- Modüllerin çalıştırılma sırası değişebilir.
- Hiç `require` edilmeyen modüller hiç eval edilmeyebilir.
- Side effect barındıran modüllerde beklenmedik davranışlara yol açabilir.

Çözüm olarak, inlining'i tamamen kapatabilir veya yalnızca belirli modülleri hariç tutabilirsiniz:

```
// metro.config.js
module.exports = {
  transformer: {
```

```

    async getTransformOptions() {
      return {
        transform: {
          inlineRequires: false,
        },
      };
    },
  },
};

```

veya daha ince ayar:

```

module.exports = {
  transformer: {
    async getTransformOptions() {
      return {
        transform: {
          inlineRequires: {
            blockList: {
              [require.resolve('./src/DoNotInlineHere.js')]: true,
            },
          },
          nonInlinedRequires: [
            'react', // require('react') hiçbir yerde inlined olmasın
          ],
        },
      };
    },
  },
};

```

React Native’de Zamanlayıcılar (Timers)

React Native, tarayıcı ortamındaki zamanlayıcı API’lerini büyük ölçüde aynı şekilde sunar. Temel zamanlayıcı fonksiyonları şunlardır:

- `setTimeout` / `clearTimeout`
- `setInterval` / `clearInterval`
- `setImmediate` / `clearImmediate`
- `requestAnimationFrame` / `cancelAnimationFrame`

Bu fonksiyonlar, JS event loop’u ve React Native bridge mekanizması ile birlikte çalışarak, farklı zamanlama ihtiyaçlarına cevap verir.

`requestAnimationFrame` vs. `setTimeout(fn, 0)`

`requestAnimationFrame(fn)`

- Bir sonraki *paint/frame* öncesinde çalıştırılmak üzere planlanır.
- Ekran yenileme döngüsüne senkron olduğu için, animasyon ve frame’e bağlı hesaplamalar (örneğin konum/görsel güncelleme) için idealdir.
- Amaç: “Bir sonraki çizimden hemen önce bu işi yap” demektir.

`setTimeout(fn, 0)`

- “Bu blok biter bitmez, mümkün olan en kısa sürede çalıştır” gibi düşünülebilir.
- Frame’e senkron değildir; yalnızca bir sonraki zamanlama aralığında, JS boşalınca tetiklenir.
- Modern cihazlarda çok sık tetiklenebilir; örneğin iPhone 5S gibi bir cihazda saniyede 1000+ kez bile çağrılması mümkün olabilir.

Özetle:

- `requestAnimationFrame`: frame’e *senkron*, çizim öncesi animasyon/hesaplama için uygundur.
- `setTimeout(fn, 0)`: sadece “hemen sonra” çalışır; fakat ekranın çizim ritmiyle doğrudan bağlantısı yoktur.

setImmediate Nedir?

setImmediate, içinde bulunduğunuz JS bloğunun çalışması tamamlandıktan hemen sonra, native tarafa gönderilecek batched response'tan önce çalıştırılır. Yani:

- Mevcut senkron JS işi bittiğinde,
- Native katmana kontrollü bir geçiş yapılmadan hemen önce devreye girer.

Önemli bir detay:

- **setImmediate** içinden tekrar **setImmediate** çağırırsanız, araya native taraf girmeden ardışık şekilde JS içinde çalışmaya devam edebilir.

React Native'in *Promise* implementasyonu, asenkronluk sağlamak için içerde **setImmediate** benzeri mekanizmalar kullanır; böylece “bir sonraki tick” mantığıyla, mevcut çağrı yığını boşaldıktan sonra çalışacak işler planlanmış olur.

Bu kısa not, React Native'de farklı zamanlayıcıların *ne zaman* tetiklendiğini zihinde netleştirmek ve özellikle animasyon ile genel asenkron işler arasında doğru aracı seçmek için bir referans olarak kullanılabilir.

React Native Codegen

1. Codegen'in İşlevi

- Codegen, **modül ve bileşen arayüzlerinin TypeScript/Flow ile tanımlanması ve bu tanımlardan hareketle köprü (bridge) katmanının otomatik olarak üretilmesini** sağlayan bir araçtır.
- Geliştirici yalnızca bir *spec* (sözleşme) dosyası oluşturur; örneğin:
 - Modül adı: `MathModule`,
 - Fonksiyon imzaları: `add(a, b): number` vb.
- Codegen, bu tip tanımlarını esas alarak:
 - Android ve iOS için native modül iskeletlerini,
 - Ortak C++ köprü (glue) katmanınıderleme aşamasında otomatik biçimde üretir.

2. Gereklilik Gerekçesi

- Codegen kullanılmadığı durumda:
 - Aynı modül arayüzünün JavaScript, Android ve iOS kod tabanlarında **ayrı ayrı ve manuel** olarak tanımlanması gerekmektedir.
 - Herhangi bir imza veya tip değişikliğinde, bu değişikliğin üç farklı platformda da tutarlı biçimde güncellenmesi zorunlu hâle gelir; bu da bakım maliyetini ve hata olasılığını artırır.
- Codegen kullanıldığında:
 - Modül arayüzü için **tek bir hakikat kaynağı** (TypeScript/Flow spec dosyası) oluşturulmuş olur.
 - Android ve iOS köprü katmanları, bu merkezi tip tanımından otomatik olarak türetildiği için, tip uyumsuzluğu, imza farklılığı veya yazım hatası kaynaklı sorunlar önemli ölçüde azaltılır.

3. Örnek Spec Tanımı (JS/TS Tarafı)

```
// MathModuleSpec.ts

export interface Spec {
  add(a: number, b: number): number;
  multiply(a: number, b: number): number;
}
```

- Bu dosya, ilgili modülün hangi fonksiyonları sunduğunu ve bu fonksiyonların parametre ile dönüş tiplerini tanımlayan bir **arayüz sözleşmesi** görevi görmektedir.
- Derleme süreci sırasında Codegen, söz konusu spec dosyasını işleyerek native tarafta modüle ilişkin iskelet sınıfları ve köprü kodlarını üretir.
- Geliştiricinin sorumluluğu, native kod tarafında yalnızca **a + b** veya **a * b** gibi **iş mantığını** gerçekleştiren gövdeleri yazmakla sınırlı kalır; köprüleme, tip eşleştirme ve imza uyumluluğu gibi yapısal ayrıntılar Codegen tarafından üstlenilir.

React Native Headless JS

1. Headless JS nedir?

- **Headless JS** = Uygulama arka plandayken JavaScript çalıştırma mekanizmasıdır.
- Tipik kullanım senaryoları:
 - Arka planda veri senkronizasyonu yapmak,
 - Push bildirimi geldiğinde belirli bir işlem gerçekleştirmek,
 - Kullanıcı arayüzü kapalıyken dahi müzik/medya çalmaya devam etmek.
- Temel kural: Headless JS içinde **UI'ya dokunulmaz**; ekran değiştirme, component render etme gibi işlemler yapılmaz, yalnızca arka plan işleri yürütülür.

2. JS tarafı: Task tanımlama

Headless JS task'i JavaScript tarafında `AppRegistry` üzerinden kaydedilir:

```
import { AppRegistry } from 'react-native';

AppRegistry.registerHeadlessTask('SomeTaskName', () =>
  require('./SomeTaskName'),
);
```

İlgili dosyada (örneğin `SomeTaskName.js`) asenkron task tanımı yapılır:

```
module.exports = async (taskData) => {
  // network isteği, timer, log vb.
  // Dikkat: UI yok!
};
```

- Task tamamlandığında (Promise resolve olduğunda), React Native JS motoru tekrar “uyku” durumuna geçer.

3. Native tarafı (Android Service) ana fikir

Android tarafında, Headless JS task'ini başlatmak için `HeadlessJsTaskService` sınıfından türetilen küçük bir servis yazılır:

```
public class MyTaskService extends HeadlessJsTaskService {
    @Override
    protected @Nullable HeadlessJsTaskConfig getTaskConfig(Intent intent) {
        Bundle extras = intent.getExtras();
        if (extras != null) {
            return new HeadlessJsTaskConfig(
                "SomeTaskName",           // JS'te register edilen isim
                Arguments.fromBundle(extras), // taskData
                5000,                       // timeout (ms)
                false                       // foreground'da çalışsın mı?
            );
        }
        return null;
    }
}
```

Manifest'e servis tanımı eklenir:

```
<service android:name=".MyTaskService" />
```

Daha sonra native tarafta bu servis başlatıldığında:

```
Intent service = new Intent(getApplicationContext(), MyTaskService.class);
Bundle bundle = new Bundle();
bundle.putString("foo", "bar");
service.putExtras(bundle);

getApplicationContext().startForegroundService(service);
```

JS motoru ayağa kalkar, `SomeTaskName` task'i çalıştırılır ve iş bittikten sonra tekrar durdurulur.

4. Retry (yeniden deneme) mekanizmasının özü

- Varsayılan olarak Headless JS task'leri **otomatik retry** yapmaz.
- Yeniden deneme davranışı isteniyorsa:
 - Native tarafta `HeadlessJsRetryPolicy` (örneğin `LinearCountingRetryPolicy(3, 1000)`) tanımlanır,
 - JS tarafında hata durumunda özel bir hata fırlatılır.

JS tarafında örnek kullanım:

```
import { HeadlessJsTaskError } from 'HeadlessJsTask';

module.exports = async (taskData) => {
  const ok = /* ... */;
  if (!ok) {
    throw new HeadlessJsTaskError(); // retry tetikler
  }
};
```

5. Dikkat edilmesi gereken temel noktalar

- Headless task, varsayılan konfigürasyonda **foreground** durumdayken çalıştırılırsa uygulama crash edebilir; bu davranış ancak açıkça `allowInForeground = true` ile değiştirilir (performans açısından genellikle önerilmez).
- `BroadcastReceiver` üzerinden başlatılıyorsa, cihazın uykuya geçmesini engellemek için `HeadlessJsTaskService.acquireWakeLockNow(context)` çağrısı yapılmalıdır.
- Tipik kullanım: Örneğin ağ bağlantısı yeniden geldiğinde, uygulama arka plandayken sessizce veri senkronizasyonu yapmak gibi senaryolardır.

6. “Bu npm kütüphaneleriyle yapılamaz mıydı?” sorusu

Bu soru iki yönlü cevaplanabilir:

Evet: Çoğu durumda npm kütüphaneleri bu işi sizin yerinize yapar.

- Örneğin:
 - `react-native-background-fetch`,
 - `react-native-push-notification`,
 - `react-native-track-player`

gibi kütüphaneler:

- Android tarafında Headless JS + native service,
- iOS tarafında ilgili background modlarını (BGTask, background fetch, push, audio vb.)

sizin adınıza kullanır.

Geliştirici yalnızca:

```
BackgroundFetch.configure(...);  
TrackPlayer.setupPlayer(...);
```

gibi JS API'lerini çağırırken, kütüphane yazarı arka planda: `HeadlessJsTaskService`, `HeadlessJsTaskConfig` ve `AppRegistry.registerHeadlessTask(...)` gibi detayları halletmiş olur. Bu nedenle, sanki yalnızca “npm paketiyle çözülmüş” gibi görünür; aslında bu paketler JS + native + Headless JS kombinasyonudur.

Hayır: Saf npm (sadece JS) ile tam arka plan işi yapılamaz.

- JS tarafındaki `setTimeout`, `setInterval`, `Promise`, `async/await` gibi mekanizmalar, yalnızca uygulama foreground'dayken ve JS engine çalışırken etkindir.
- Uygulama arka plana geçtiğinde, hatta tamamen kapatıldığında veya işletim sistemi tarafından sonlandırıldığında:
 - “Her 15 dakikada bir sync yap”,
 - “Ağ bağlantısı gelince sessizce veri çek”,
 - “WiFi değişince otomatik bir işlem yap”

gibi senaryolar yalnızca saf JS ile gerçekleştirilemez; bu noktada:

- Android'de Headless JS + Service,
- iOS'ta background modes + native kod

devreye girer.

7. Headless JS ne zaman gerçekten gerekli olur?

Aşağıdaki durumlarda hazır npm paketleri yetersiz kalabilir ve doğrudan Headless JS kullanımı gündeme gelebilir:

1. Çok spesifik bir native SDK'nın sarmalanması gerektiğinde (örneğin yalnızca Android'e özgü bir telekom SDK'sı ve belirli event'lerde özel arka plan task'leri tetiklenmesi isteniyorsa),
2. Mevcut kütüphanelerin sağladığı davranışın ihtiyaçları tam olarak karşılamadığı, özel "background sync" senaryolarının yazılmak istendiği durumlarda,
3. Kendi React Native kütüphanenizi geliştirdiğiniz ve diğer projelerin de npm üzerinden kullanacağı bir çözüm sunduğunuz durumlarda.

Bu tür durumlarda, native tarafta `MyTaskService extends HeadlessJsTaskService` ve JS tarafında `AppRegistry.registerHeadlessTask` kullanarak kendi arka plan işçinizi oluşturursunuz.

Sonuç

Bu notlarda ele alınan başlıklar, React Native geliştirme sürecinin farklı katmanlarına dağılmış gibi görünse de ortak bir ekseninde buluşmaktadır: *kontrol* ve *farkındalık*. Metro ve `metro.config.js` ile paketleme sürecinin nasıl çalıştığını anlamak, Gesture Responder System ve jest kütüphaneleriyle etkileşim modelini bilinçli kurmak, mobil uygulama güvenliği tarafında hangi veriyi nerede ve nasıl sakladığınıza dikkat etmek, performans ve build süreleri için JS ve native dünyasının sınırlarını doğru çizmek; hepsi aynı hedefe hizmet eder: daha öngörülebilir, daha bakımı kolay ve kullanıcı açısından daha güvenilir bir uygulama ortaya koymak.

Özellikle güvenlik, performans ve arka plan işlemleri (Headless JS, zamanlayıcılar, lazy yükleme, FlatList ayarları, build optimizasyonları vb.) birer “sonradan bakılacak detay” değil, tasarımın ilk aşamalarında düşünülmesi gereken temel kısıtlardır. Bu dokümandaki özetlerin, yeni bir ekran tasarlarken, bir API entegrasyonu yaparken veya projeyi ölçeklendirirken küçük ama kritik kararları daha bilinçli almanıza yardımcı olması hedeflenmiştir. Son kertede, React Native ekosisteminde güçlü uygulamalar geliştirmek, yalnızca doğru kütüphaneleri seçmekten değil; bu tür altyapısal kavramları cebinizde taşıyıp günlük pratiğinize yedirmenizden geçer.